

Rapport de Projet Android

Programmation Mobile — Communication Bluetooth

Aymen ACHAGUI & Ibrahim KHELIL | 2SN - R1 — ENSEEIHT 2025-2026

1. Travail réalisé

L'objectif du projet est de développer une application Android permettant à deux smartphones de communiquer via Bluetooth pour contrôler une maison connectée simulée. L'un des téléphones joue le rôle de **serveur (hub)** et communique avec l'API REST, tandis que l'autre joue le rôle de **client (télécommande)**.

Les fonctionnalités implémentées sont les suivantes :

- Sélection du rôle au lancement : serveur (hub) ou client (télécommande)
- Établissement d'une connexion Bluetooth entre les deux téléphones via RFCOMM
- Récupération périodique des appareils connectés (GET toutes les 10 secondes) via Volley
- Affichage de la liste des appareils avec leur état, marque, modèle et données
- Contrôle ON/OFF des appareils : le serveur fait le POST REST, le client envoie une commande **TOGGLE** via Bluetooth
- Envoi automatique des données JSON au client après chaque mise à jour
- Gestion du cycle de vie : arrêt des tâches périodiques lors de la mise en pause

2. Architecture de l'application

2.1 Vue d'ensemble

L'application est composée de deux activités et de plusieurs classes internes de type Thread :

TABLE 1 – Description des composants de l'application et de leurs rôles

Composant	Description du rôle
MainActivity	Écran d'accueil : choix du rôle et initialisation de la connexion Bluetooth.
SecondActivity	Affichage de la liste des appareils, gestion des requêtes REST et du flux Bluetooth.
AcceptThread	Thread serveur : écoute et accepte les connexions entrantes (<code>accept()</code>).
ConnectThread	Thread client : initie la connexion active vers le serveur (<code>connect()</code>).
ConnectedThread	Thread de communication : gère les lectures et écritures continues sur le socket BT.

2.2 MainActivity

Cette activité constitue l'écran d'accueil. Elle propose deux boutons permettant de choisir le rôle du téléphone. Selon le choix :

- **Mode Serveur** : lance `AcceptThread` qui ouvre un port d'écoute RFCOMM identifié par un UUID unique. La méthode `accept()` est bloquante, d'où l'exécution dans un thread séparé.
- **Mode Client** : lance `ConnectThread` qui récupère le premier appareil Bluetooth appairé et appelle `connect()` avec le même UUID. Cette méthode est également bloquante.

Une fois la connexion établie des deux côtés, la méthode `manageConnectedSocket()` sauvegarde le socket dans une variable statique et lance `SecondActivity` via un Intent explicite.

2.3 SecondActivity

Cette activité gère l'affichage et la logique métier. Son comportement diffère selon le rôle :

- **Serveur** : initialise Volley, lance un Handler périodique (toutes les 10s) qui fait un GET sur l'API REST. La réponse JSON est affichée localement et envoyée au client via Bluetooth.
- **Client** : attend passivement les données JSON envoyées par le serveur. À réception, il met à jour son affichage.

Le `ConnectedThread` tourne en continu et lit l'`InputStream` du socket BT. Les messages sont délimités par `\n`. À réception d'un message complet, il l'envoie au `bluetoothHandler` qui s'exécute sur le thread UI pour mettre à jour l'affichage en toute sécurité.

2.4 Cycle de vie

Les méthodes `onResume()` et `onPause()` gèrent le Handler périodique côté serveur : il est démarré à l'affichage de l'activité et arrêté lorsqu'elle passe en arrière-plan, conformément aux bonnes pratiques Android.

3. Fonctionnement de l'application

3.1 Scénario d'utilisation

1. Les deux téléphones sont appairés manuellement via les paramètres Bluetooth.
2. Sur le téléphone serveur : appui sur « Lancer le Serveur » → `AcceptThread` attend.
3. Sur le téléphone client : appui sur « Lancer le Client » → `ConnectThread` se connecte.
4. La connexion est établie → les deux téléphones passent à `SecondActivity`.
5. Le serveur interroge l'API REST et envoie la liste des appareils au client via BT.
6. Le client appuie sur un bouton ON/OFF → envoie `"TOGGLE:id"` au serveur via BT.
7. Le serveur reçoit la commande, fait le POST REST, rafraîchit et renvoie les données mises à jour.

3.2 Communication Bluetooth

La communication repose sur un socket RFCOMM. Les messages sont des chaînes de caractères terminées par `\n` (délimiteur). Le `ConnectedThread` utilise un `StringBuilder` pour accumuler les données reçues fragment par fragment et n'extraire que les messages complets.

- **Serveur** → **Client** : tableau JSON complet des appareils (après chaque GET ou POST)

- **Client** → **Serveur** : commande texte "TOGGLE:id" pour allumer/éteindre un appareil

4. Choix de conception et difficultés rencontrées

4.1 Choix de conception

- **Socket statique** : le `BluetoothSocket` est déclaré `static` dans `MainActivity` pour être accessible directement par `SecondActivity` sans passer par un `Intent` (les `Intents` ne supportent pas les objets non-sérialisables).
- **Délimiteur \n** : les messages JSON peuvent être volumineux et arriver en plusieurs fragments réseau. Le délimiteur permet de reconstituer les messages complets côté récepteur.
- **Handler périodique** : l'utilisation de `postDelayed()` plutôt qu'un `Thread` dédié simplifie la gestion du cycle de vie et évite les problèmes de synchronisation avec le thread UI.
- **UUID personnalisé** : un UUID unique a été généré pour identifier notre application et éviter les conflits avec d'autres applications Bluetooth.

4.2 Difficultés rencontrées

- **Permissions Bluetooth** : l'application crashait au démarrage sur Android 10 car les permissions `BLUETOOTH` et `BLUETOOTH_ADMIN` manquaient dans le Manifest. Après analyse des logs Logcat, les permissions ont été ajoutées correctement. `Theme.AppCompat.Light.NoActionBar`.
- **Fragmentation des messages BT** : les données JSON étant volumineuses, elles arrivent parfois en plusieurs paquets. L'utilisation d'un `StringBuilder` avec détection du délimiteur `\n` a résolu ce problème.
- **Thread UI** : les données reçues dans `ConnectedThread` ne pouvaient pas modifier directement l'interface. L'utilisation du `Handler` avec `Looper.getMainLooper()` a permis de transférer le traitement vers le thread UI.